

POCKETSMART AI

Project Team

TEAM LEADER

ROHITH S

NM ID: 4B2937F3713DEDE9E6A4941AEABE84A1

TEAM MEMBER

CHANDRU T

NM ID: 907B183358FBBB99AF14E7F18D917DC9

TEAM MEMBER

HEEDHESH R

NM ID: 40D4B6DFD412A2597FC4FADD8710CC96

TEAM MEMBER

ASHWIN S

NM ID: 446946BAC63A2BEA45F45AA4CA3D3B47

POCKETSMART AI

AI-Powered Smart Planning & Recommendation Web Application

PROJECT DOCUMENTATION

Prepared from the submitted PocketSmart AI project source code

September 2026

1. Abstract

PocketSmart AI is a web-based smart planning application designed to help users generate practical recommendations for different personal planning scenarios. The project provides a browser-based interface with user authentication, planning modules, recommendation generation, and history management. The application is implemented as a Python web application and uses an AI integration where configured to produce more personalized recommendations.

The system is organized around separate planning experiences such as Home, Party, and Jewelry planning. Users can enter relevant preferences and receive recommendations through the application interface. The project also maintains user-related state and history so that previous planning activity can be accessed again.

2. Introduction

Planning purchases, events, and lifestyle requirements can involve comparing many options while staying within a budget or preference set. PocketSmart AI attempts to simplify this process by collecting user requirements through focused forms and converting them into useful recommendations.

The project combines a conventional web stack with Python server-side processing and optional generative-AI functionality. This makes it suitable as an academic full-stack/AI project demonstrating frontend design, backend routing, authentication, data handling, and API integration.

3. Problem Statement

Users often need help selecting suitable products, arrangements, or ideas according to requirements such as category, occasion, preferences, and budget. Manual research can be time-consuming and may produce scattered results. A centralized recommendation system can collect these requirements and return structured suggestions through a single interface.

4. Objectives

- Provide a simple web interface for smart planning
- Collect user requirements through dedicated planning forms
- Generate recommendations based on the submitted inputs
- Support authenticated user sessions
- Preserve useful planning history
- Integrate generative AI when the configured AI service is available
- Demonstrate practical full-stack application development

5. Key Features

- User registration/login and logout flow
- Home planning module
- Party planning module
- Jewelry planning module
- Recommendation generation
- History access for previous planning activity
- Session-based user state
- Responsive web interface using HTML/CSS/JavaScript
- Python/Flask backend routing and processing
- AI integration support

6. Technology Stack

Backend	Python with Flask (detected in source code)
Frontend	HTML, CSS and JavaScript
AI	Google Gemini / generative AI integration is present in the project source
Data/State	SQLite/data persistence references are present
Authentication	Login/logout and session-related functionality are present

7. System Architecture

The application follows a client-server architecture. The browser provides the presentation layer. User requests are sent to the Python backend, where routes process input, apply application logic, interact with storage/services, and return HTML or generated results. Where configured, an AI service is called to create recommendation content.

Flow: User → Web Interface → Flask Routes → Application Logic → Storage / AI Service → Recommendation Result → User Interface

8. Module Description

8.1 Authentication Module

Handles user access through registration/login-related functionality and maintains session information during use. Logout ends the authenticated session.

8.2 Home Planner

Provides a planning interface for home-related requirements. User inputs are collected and processed to produce suitable recommendations.

8.3 Party Planner

Helps users plan party-related requirements by collecting event preferences and generating recommendations based on the submitted details.

8.4 Jewelry Planner

Provides a dedicated planning flow for jewelry-related requirements, allowing recommendations to be generated from user-selected criteria.

8.5 AI Recommendation Module

The AI layer converts structured user requirements into recommendation content. When an AI provider/API key is configured, the backend can send an appropriate prompt and process the generated response.

8.6 History Module

Stores or displays previous planning activity so users can revisit earlier recommendations and inputs.

9. Typical Workflow

1. User opens PocketSmart AI
2. User creates an account or logs in
3. User selects the required planner
4. User enters preferences and requirements
5. Backend validates and processes the input
6. Recommendation logic/AI generates the result
7. Result is displayed in the browser
8. Relevant activity can be stored in history
9. User can return later and review previous planning activity

10. Project Structure

The submitted project contains Python backend code together with frontend templates/assets. The exact file names may vary with the submitted ZIP structure.

```
pocketsmart/.env.example
pocketsmart/.gitignore
pocketsmart/README.md
pocketsmart/app.py
pocketsmart/models/__init__.py
pocketsmart/requirements.txt
pocketsmart/services/__init__.py
pocketsmart/services/ai_engine.py
pocketsmart/services/mock_ai.py
pocketsmart/static/style.css
pocketsmart/templates/base.html
pocketsmart/templates/dashboard.html
pocketsmart/templates/history.html
pocketsmart/templates/index.html
pocketsmart/templates/login.html
pocketsmart/templates/planner_home.html
pocketsmart/templates/planner_jewelry.html
pocketsmart/templates/planner_party.html
pocketsmart/templates/register.html
pocketsmart/templates/results_home.html
pocketsmart/templates/results_jewelry.html
pocketsmart/templates/results_party.html
```

11. Installation and Setup

Recommended setup for local development:

1. Install Python 3.x
2. Extract the PocketSmart AI project
3. Open a terminal in the project directory
4. Create and activate a virtual environment:

```
python -m venv venv
venv\Scripts\activate      (Windows)
source venv/bin/activate   (Linux/macOS)
```

5. Install dependencies from the project's requirements file, if included:

```
pip install -r requirements.txt
```

6. Configure required environment variables/API keys according to the project's configuration
7. Start the Flask application using its configured entry point
8. Open the local URL shown by Flask in a web browser

12. Configuration & Security

- API keys and secret credentials should be kept outside publicly shared source code
- Environment variables or a local .env configuration should be used where supported
- Debug mode should be disabled in production
- Passwords should be stored using secure hashing rather than plain text
- Input validation should be applied to user-provided values

13. Testing

Suggested functional tests include: registration with valid and invalid data; login/logout; opening each planner; submitting valid planner inputs; handling missing inputs; generating recommendations with and without AI configuration; saving and viewing history; refreshing authenticated pages; and checking unauthorized access to protected routes.

Expected result: valid inputs should produce the intended planning/recommendation response, while invalid or incomplete inputs should be handled gracefully without exposing application errors or credentials.

14. Advantages

- Centralized planning experience
- User-focused recommendation workflow
- Modular planner design
- AI can make recommendations more personalized
- Demonstrates frontend and backend integration
- Can be extended with additional planning categories and services

15. Limitations

- AI recommendations depend on the configured AI service and its availability
- Generated recommendations should be treated as suggestions rather than guaranteed or professional advice
- Production deployment requires stronger security, validation, monitoring, and scalable data storage
- Recommendation quality depends on the quality and completeness of user inputs

16. Future Enhancements

- Add more planning categories
- Add richer user profiles and preference learning
- Add budget tracking and cost comparison
- Add recommendation feedback and rating mechanisms for system improvement
- Add image-based recommendations where appropriate
- Add downloadable/shareable plans
- Deploy using a production WSGI server and managed database
- Add automated tests and CI/CD

17. Conclusion

PocketSmart AI demonstrates how a Python web application can combine structured user input, modular planning features, authentication, history, and generative AI to create a practical recommendation platform. The project provides a strong foundation for further development into a more comprehensive personalized planning system.

18. Viva / Interview Questions

1. Why did you choose Flask for the backend?
2. How does the login/session mechanism work?
3. How are planner inputs sent from frontend to backend?
4. How is AI integrated into the application?
5. How do you protect API keys?
6. How is recommendation history stored?
7. What happens when the AI API is unavailable?
8. How would you deploy this project?
9. What security improvements are needed for production?
10. How could the recommendation system be improved in future?

Appendix A. Application Screenshots

The following screenshots demonstrate the working application: the development environment, account creation, the dashboard, and the Home Planner producing an AI-generated budget plan.

A.1 Development Environment

The project's source structure, styling variables, and the build assistant's setup summary inside the IDE.

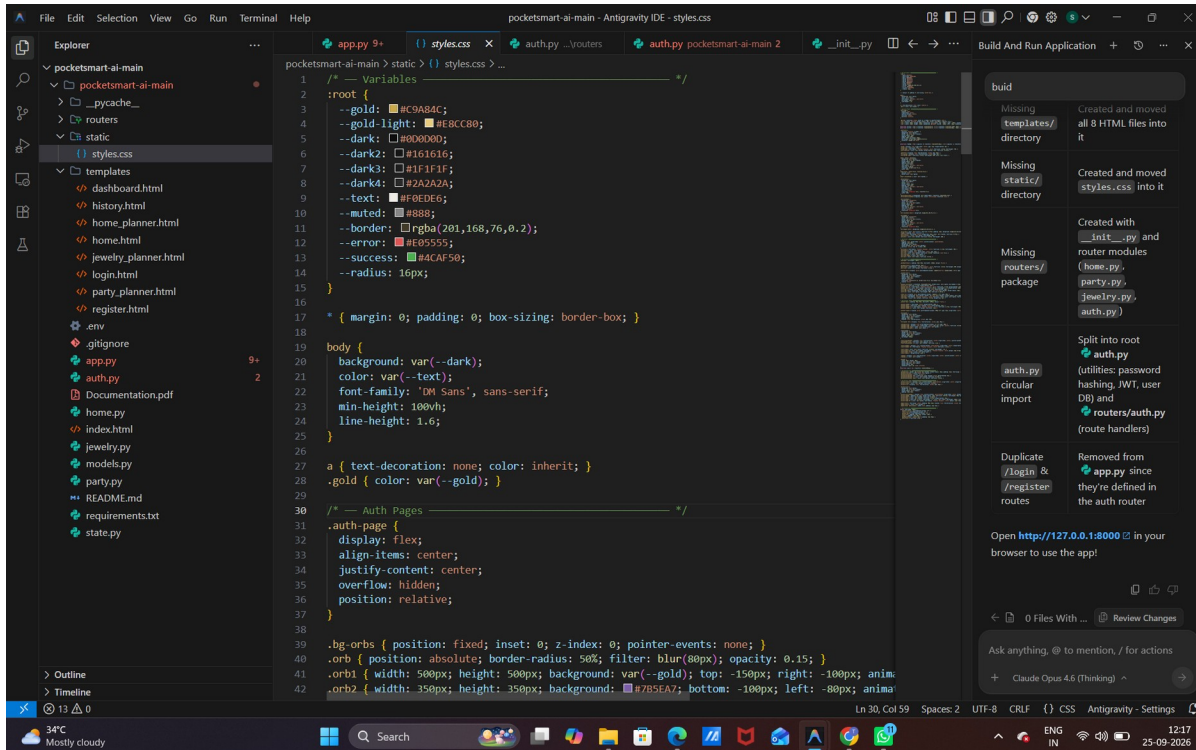


Fig. A.1 — Project source tree and styles.css in the Antigravity IDE

A.2 Account Creation

The registration screen where a new user creates a PocketSmart AI account.

Fig. A.2 — Create Account page

A.3 Dashboard

The main dashboard after login, showing recommendation stats and the three smart budget planners.

Fig. A.3 — Dashboard with Home, Party and Jewelry planner cards

A.4 Home Planner — Requirements Form

The Home Interior Budget Planner form, where the user enters budget, rooms, style and preferred shopping platforms.

PocketSmart.AI

Dashboard Home Party Jewelry History Itz me Logout

Enter Your Requirements

Total Budget (₹) *

10000

Number of Rooms *

2 Rooms

Interior Style *

Bohemian

Rooms to Furnish

☒ Living Room ☐ Bedroom

☒ Kitchen ☒ Bathroom

☒ Dining Room ☐ Home Office

Preferred Platforms

☒ Amazon ☒ Flipkart

☒ IKEA ☐ Pepperfry

Additional Preferences

e.g. prefer light colors, need storage solutions...

+ Generate AI Recommendations

Your Personalized Budget Plan Will Appear Here

Fill in your budget, rooms and style on the left, then click Generate to get a structured plan with product tables and direct shopping links.

34°C Mostly cloudy

Search

ENG IN 12:08 25-09-2026

Fig. A.4 — Home planner input form, before generating recommendations

A.5 Home Planner — AI-Generated Plan

The resulting AI-generated budget plan, with a budget summary and itemized, categorized product recommendations with shopping links.

The screenshot displays the PocketSmart AI Home Interior Planner interface. On the left, the 'Enter Your Requirements' section includes input fields for 'Total Budget (₹)' (10000), 'Number of Rooms' (2 Rooms), and 'Interior Style' (Bohemian). It also features checkboxes for 'Rooms to Furnish' (Living Room, Kitchen, Dining Room, Bedroom, Bathroom, Home Office) and 'Preferred Platforms' (Amazon, Flipkart, IKEA, Pepperfry). An 'Additional Preferences' field is set to 'no'. A '+ Generate AI Recommendations' button is at the bottom of this section.

The right side of the interface shows the 'Your Personalized Budget Plan' for ₹10,000 across 2 rooms in a Bohemian style. Below this is a 'Budget Summary' table:

TOTAL BUDGET	ALLOCATED	REMAINING	CATEGORIES
₹10,000	₹10,000	₹0	4

Below the budget summary is a section titled 'Boho Textiles & Furnishings' with a total value of ₹3,500. It contains a table of recommended items:

ITEM	DESCRIPTION	PRICE	QTY	SHOPPING LINKS
Macrame Cushion Covers (Pack of 2)	Boho tassel-fringed cotton cushion covers for living room seating	₹450	x2	Amazon , Flipkart , IKEA
Natural Jute Floor Runner	Textured woven runner for the dining/living hallway transition	₹1,200	x1	Amazon , Flipkart , IKEA
Boho Aztec Dining Table Runner	Woven geometric pattern runner for dining table accent	₹400	x1	Amazon , Flipkart , IKEA
Patterned Fabric Shower Curtain	Boho print waterproof shower drape with curtain hooks	₹1,000	x1	Amazon , Flipkart , IKEA

The interface also includes a top navigation bar with links to Dashboard, Home, Party, Jewelry, History, and a user profile section with 'its me' and 'Logout' options. The bottom of the screen shows a Windows taskbar with the date and time (12:08, 25-09-2026).

Fig. A.5 — Generated budget plan with itemized shopping recommendations

Project Links

GitHub Repository: <https://github.com/mrchandrut-coder/PocketSmart-AI-Your-Smart-Budget-Recommendation-Assistant>

Demo Link: <https://drive.google.com/file/d/1xzyRepwGcTW3uf0zOxxIdIfaH2ytCTTD/view?usp=drivesdk>